

오프라인

2026.06.24 WED

파이썬 프로그래밍 2일차 강의자료

반복문 · 리스트와 튜플 · 딕셔너리와 집합

오늘의 한 줄 목표

반복문과 자료구조로 여러 개의 데이터를 한꺼번에 다루는 프로그램을 만든다.

대상: 비전공자 1~4학년 · 구성: 이론 + 실습

일정



오늘의 시간표

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

교시	주제	이론	실습
1	for 반복문 & range()	반복·for·range·중첩	구구단·합계·별찍기
2	while & 반복 제어	while·break·continue	숫자 맞추기 게임
3	리스트	인덱싱·슬라이싱·메서드	점수 통계·To-do
4	리스트 심화 & 튜플	컴프리헨션·2차원·튜플	로또 생성기
5	딕셔너리 & 집합	키-값·set 연산	단어장·빈도·공통수강생

오늘의 전환

- 어제는 한 번 실행, 오늘은 여러 번 처리
- 반복문으로 흐름을 자동화
- 자료구조로 여러 데이터를 한꺼번에 관리

TAKEAWAY

오늘의 목표: 반복문과 자료구조로 여러 개의 데이터를 한꺼번에 다룬다.

복습

어제 복습 & 오늘의 연결고리

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

어제의 한계

- 변수·입출력·연산자·조건문으로 한 번 판단하는 프로그램을 만들었습니다.
- 하지만 자판기처럼 한 번 실행하면 프로그램이 끝났습니다.
- 복사-붙여넣기는 느리고, 실수하고, 수정하기 어렵습니다.

오늘의 해결

- 반복문으로 여러 번 처리합니다.
- 리스트와 튜플로 여러 값을 묶습니다.
- 딕셔너리와 집합으로 이름표·중복·공통값을 다룹니다.

끝나면 만들 수 있는 것

- 구구단·별찍기 자동 출력, 숫자 맞추기 게임, 성적 통계, 단어장, 연락처 관리

TAKEAWAY

복붙 대신 반복문, 변수 여러 개 대신 자료구조로 갑니다.

1교시

1교시 — for 반복문 & range()

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: 반복되는 작업을 for 반복문으로 자동화한다.

이론 20분

실습 30분

휴식 10분

이론

반복이 필요한 이유

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- "안녕하세요"를 100번 출력하려면?
- 복사-붙여넣기는 한계가 있습니다 (느리고, 실수하고, 못 바꿈)
- 컴퓨터가 가장 잘하는 일 = 반복
- 그 도구가 바로 반복문(Loop)

Python example

```
print("안녕하세요")  
print("안녕하세요")  
# ... 98줄 더??
```

TAKEAWAY

"안녕하세요"를 100번 출력하려면?

이론

for 반복문이란?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 정해진 횟수만큼 또는 모음(리스트 등)의 요소마다 반복
- "range(3)만큼 반복해라" → 3번 실행
- 출력:

Python example

```
for i in range(3):  
    print("안녕하세요")
```

```
안녕하세요  
안녕하세요  
안녕하세요
```

TAKEAWAY

정해진 횟수만큼 또는 모음(리스트 등)의 요소마다 반복

이론

for 기본 구조

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- for + 변수 + in + 반복대상 + 콜론 :
- 어제 배운 콜론·들여쓰기 규칙 그대로 적용
- 변수에는 반복할 값이 하나씩 들어옵니다

Python example

```
for 변수 in 반복할_것:  
    실행할 코드    # ← 들여쓰기 필수!
```

TAKEAWAY

for + 변수 + in + 반복대상 + 콜론 :

이론

range() — 숫자 범위 만들기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- range() = 연속된 숫자를 만들어주는 도구
- 0부터 시작, 5는 포함 안 됨 (0~4, 총 5개)
- 출력:

Python example

```
for i in range(5):  
    print(i)  
  
0  
1  
2  
3  
4
```

TAKEAWAY

range() = 연속된 숫자를 만들어주는 도구

이론

range(시작, 끝)

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- range(1, 6) → 1부터 6 직전(5)까지
- 시작은 포함, 끝은 불포함
- 1부터 세고 싶을 때 자주 사용

Python example

```
for i in range(1, 6):  
    print(i)    # 1 2 3 4 5
```

TAKEAWAY

range(1, 6) → 1부터 6 직전(5)까지

이론

range(시작, 끝, 간격)

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 세 번째 숫자 = 증가 폭(step)
- 2씩 증가 → 짝수만!

Python example

```
for i in range(0, 11, 2):  
    print(i)    # 0 2 4 6 8 10
```

```
for i in range(10, 0, -1):  
    print(i)    # 10 9 8 ... 1 (거꾸로)
```

TAKEAWAY

세 번째 숫자 = 증가 폭(step)

이론

반복 변수 활용하기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 변수 i 는 매 반복마다 값이 바뀜 ($1 \rightarrow 2 \rightarrow 3$)
- 그 값을 출력·계산에 활용 가능
- 출력:

Python example

```
for i in range(1, 4):  
    print(f"{i}번째 학생 출석")
```

```
1번째 학생 출석  
2번째 학생 출석  
3번째 학생 출석
```

TAKEAWAY

변수 i 는 매 반복마다 값이 바뀜 ($1 \rightarrow 2 \rightarrow 3$)

이론

누적 합 패턴 ①

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 반복 밖에서 그릇(total) 준비 → 0으로 시작
- 반복 안에서 계속 더해 쌓기
- 가장 중요한 패턴! (합계·개수 세기 등)

Python example

```
total = 0
for i in range(1, 6):
    total = total + i
print(total)    # 15 (1+2+3+4+5)
```

TAKEAWAY

반복 밖에서 그릇(total) 준비 → 0으로 시작

이론

누적 합 패턴 ② (+= 활용)

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 어제 배운 += 가 여기서 빛을 발함
- 카운트(개수 세기)도 같은 패턴:

Python example

```
total = 0
for i in range(1, 6):
    total += i    # total = total + i 와 같음
print(total)    # 15
```

```
count = 0
for i in range(10):
    count += 1
```

TAKEAWAY

어제 배운 += 가 여기서 빛을 발함

이론

리스트도 반복할 수 있다

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- range 숫자뿐 아니라 리스트 요소도 하나씩 꺼냄
- 3교시 리스트에서 자세히! (지금은 "이런 것도 된다")
- 출력:

Python example

```
fruits = ["사과", "바나나", "포도"]  
for fruit in fruits:  
    print(fruit)
```

```
사과  
바나나  
포도
```

TAKEAWAY

range 숫자뿐 아니라 리스트 요소도 하나씩 꺼냄

이론

문자열도 반복된다

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 문자열을 for에 넣으면 글자 하나씩 꺼냄
- 글자 수 세기, 검사 등에 활용
- 출력:

Python example

```
for ch in "파이썬":  
    print(ch)
```

파
이
썬

TAKEAWAY

문자열을 for에 넣으면 글자 하나씩 꺼냄

이론

중첩 for 반복문

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- for 안에 for → 이중 반복
- 바깥 1번 돌 때 안쪽이 전부 돌
- 출력:

Python example

```
for i in range(1, 3):  
    for j in range(1, 4):  
        print(f"({i}, {j})")
```

(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3)

TAKEAWAY

for 안에 for → 이중 반복

이론

중첩 for 활용 — 구구단

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 바깥 dan = 단(2단, 3단...)
- 안쪽 i = 1~9 곱하기
- 표·달력·격자 등 2차원 출력에 필수

Python example

```
for dan in range(2, 4):  
    for i in range(1, 10):  
        print(f"{dan} x {i} = {dan*i}")  
    print("---")
```

TAKEAWAY

바깥 dan = 단(2단, 3단...)

정리

for 반복문 정리

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

오늘의 결과물

- for 변수 in 범위: + 들여쓰기
- range(n) → 0~n-1, range(a, b) → a~b-1, range(a, b, step)
- 핵심 패턴: 그릇 준비 → 반복하며 쌓기(합계·개수)
- 리스트·문자열도 요소 하나씩 반복
- 중첩 for로 2차원 작업

다음 연결

- 함수(def)로 반복되는 코드를 이름 붙여 재사용
- 모듈로 남이 만든 기능 가져다 쓰기
- 문자열을 split, join으로 더 잘 다루기

TAKEAWAY

핵심은 반복해서 입력받고, 자료구조에 저장하고, 필요한 값을 분석하는 흐름입니다.

실습

구구단 출력기

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
dan = int(input("몇 단? "))
for i in range(1, 10):
    print(f"{dan} x {i} = {dan * i}")
```

```
몇 단? 7
7 x 1 = 7
7 x 2 = 14
...
7 x 9 = 63
```

실습 포인트

- 실행:

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

1~N 합계 & 짝수 합

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
n = int(input("N: "))

# 1부터 N까지 합
total = 0
for i in range(1, n + 1):
    total += i
print(f"1~{n} 합: {total}")

# 짝수만 합
even = 0
for i in range(2, n + 1, 2):
    even += i
print(f"짝수 합: {even}")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

별 찍기 (경우의 수 체험)

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
# ① 직각 삼각형
for i in range(1, 6):
    print("*" * i)
```

실습 포인트

- 도전해 보세요:
- ② 거꾸로 삼각형 (range(5, 0, -1))
- ③ 가운데 정렬 피라미드 (공백 + 별)
- ④ 숫자 삼각형 (* 대신 i)

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

휴식

휴식 10분

잠깐 멈추고, 다음 자료구조로 넘어갈 준비

10 MIN BREAK

다음 교시: while 반복문 — 횟수를 모를 때 "조건이 참인 동안" 반복!

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장해 둡니다.

2교시

2교시 — while 반복문 & 반복 제어

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: 조건 기반 반복(while)과 break/continue로 흐름을 제어한다.

이론 20분

실습 30분

휴식 10분

이론

while 반복문이란?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- for: 정해진 횟수 반복
- while: 조건이 참인 동안 계속 반복
- 예: "정답 맞힐 때까지", "그만둘 때까지" → 횟수를 모름!

Python example

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

TAKEAWAY

for: 정해진 횟수 반복

이론

while 기본 구조

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 조건이 참이면 반복, 거짓이 되면 멈춤
- 어제 if처럼 조건은 True/False

Python example

```
while 조건:  
    실행할 코드    # ← 들여쓰기  
    (조건을 바꾸는 코드) # ← 매우 중요!
```

TAKEAWAY

조건이 참이면 반복, 거짓이 되면 멈춤

이론

while 흐름 따라가기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- ①조건 검사 → 참이면 ②③ → 다시 ①... 거짓이면 종료
- count가 1→2→3→4 가 되면 조건 거짓 → 멈춤

Python example

```
count = 1
while count <= 3:  # ① 조건 검사
    print(count)    # ② 실행
    count += 1      # ③ 값 변경 → 다시 ①
```

TAKEAWAY

①조건 검사 → 참이면 ②③ → 다시 ①... 거짓이면 종료

이론

무한 루프 주의!

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- count가 영원히 1 → 조건이 항상 참 → 무한 반복
- 멈추려면 터미널에서 Ctrl + C
- while 쓸 땐 "조건을 언젠가 거짓으로 만드는 코드" 필수!

Python example

```
count = 1
while count <= 3:
    print(count)
    # count += 1 을 빠뜨리면?
```

TAKEAWAY

count가 영원히 1 → 조건이 항상 참 → 무한 반복

이론

카운터 변수 패턴

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- ① 시작값 설정 ($i = 0$)
- ② 조건 ($i < 5$)
- ③ 증가 ($i += 1$)
- for로도 가능하지만 while로 직접 제어하는 연습

Python example

```
i = 0
while i < 5:
    print(f"{i}번 반복")
    i += 1
```

TAKEAWAY

- ① 시작값 설정 ($i = 0$)

이론

for vs while — 차이

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

	for	while
언제	횟수를 알	횟수를 모름
예	5번, 리스트 전부	정답까지, 종료까지
종료	범위 끝나면 자동	조건 거짓 되면

읽는 법

- 핵심 개념을 예시로 먼저 이해합니다.
- 코드로 직접 확인하며 감을 잡습니다.

TAKEAWAY

핵심 개념을 예시로 먼저 이해합니다.

이론

for vs while — 언제 쓸까?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

핵심 개념

- for: "1~100 더하기", "리스트 전부 출력" → 횟수 정해짐
- while: "비밀번호 맞을 때까지", "메뉴에서 종료 누를 때까지" → 조건 의존

적용 질문

- 헛갈리면: 반복 횟수가 미리 정해졌나? → 예: for / 아니오: while

TAKEAWAY

for: "1~100 더하기", "리스트 전부 출력" → 횟수 정해짐

이론

break — 반복 탈출

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- break = 반복문을 즉시 빠져나감
- "찾았으니 그만", "조건 충족, 중단"에 사용

Python example

```
for i in range(1, 100):  
    if i == 5:  
        break      # 반복 즉시 종료  
    print(i)       # 1 2 3 4
```

TAKEAWAY

break = 반복문을 즉시 빠져나감

break 활용 예

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- while True + break = 종료 조건을 안에서 결정
- 메뉴·입력 반복의 단골 패턴

Python example

```
while True:          # 일부러 무한 루프
    word = input("입력(끝: q): ")
    if word == "q":
        break         # q 입력 시 탈출
    print(f"입력값: {word}")
print("종료!")
```

TAKEAWAY

while True + break = 종료 조건을 안에서 결정

이론

continue — 이번 회차 건너뛰기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- continue = 이번 반복만 건너뛰고 다음 반복으로
- break(완전 종료)와 다름!

Python example

```
for i in range(1, 6):  
    if i == 3:  
        continue # 3일 때만 건너뛴  
    print(i)      # 1 2 4 5
```

TAKEAWAY

continue = 이번 반복만 건너뛰고 다음 반복으로

이론

continue 활용 예

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 특정 조건을 제외하고 처리할 때 유용
- 어제 배운 %(나머지)와 결합

Python example

```
# 홀수만 출력
for i in range(1, 11):
    if i % 2 == 0:
        continue # 짝수면 건너뛴
    print(i)      # 1 3 5 7 9
```

TAKEAWAY

특정 조건을 제외하고 처리할 때 유용

이론

break vs continue 한눈에

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

	break	continue
동작	반복 완전 종료	이번 회차만 건너뛴
이후	반복문 밖으로	다음 반복 계속
비유	영화관 나가기	이 장면만 스킵▶▶

읽는 법

- 핵심 개념을 예시로 먼저 이해합니다.
- 코드로 직접 확인하며 감을 잡습니다.

TAKEAWAY

핵심 개념을 예시로 먼저 이해합니다.

이론

while + 누적 패턴

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 입력을 계속 받아 쌓다가 특정 값에서 종료
- 가계부·점수 입력 등 현실 프로그램의 기본 골격

Python example

```
total = 0
while True:
    n = int(input("숫자(0이면 종료): "))
    if n == 0:
        break
    total += n
print(f"합계: {total}")
```

TAKEAWAY

입력을 계속 받아 쌓다가 특정 값에서 종료

이론

반복문의 else 절

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 반복이 break 없이 끝까지 돌면 else 실행
- (break로 중단되면 else는 건너뛴)
- 알아두면 좋은 문법 (자주는 아님)

Python example

```
for i in range(1, 4):  
    print(i)  
else:  
    print("반복 정상 종료!")
```

TAKEAWAY

반복이 break 없이 끝까지 돌면 else 실행

정리

while 흔한 실수 정리

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

오늘의 결과물

- 조건 갱신 누락 → 무한 루프 (가장 흔함!)
- 시작값을 안 정함 (count 없이 사용)
- break 위치 잘못 → 너무 일찍/늦게 종료
- =와 == 혼동 (조건에서)
- 무한 루프 시: Ctrl + C

다음 연결

- 함수(def)로 반복되는 코드를 이름 붙여 재사용
- 모듈로 남이 만든 기능 가져다 쓰기
- 문자열을 split, join으로 더 잘 다루기

TAKEAWAY

핵심은 반복해서 입력받고, 자료구조에 저장하고, 필요한 값을 분석하는 흐름입니다.

실습

숫자 맞추기 게임

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
import random
answer = random.randint(1, 100)

while True:
    guess = int(input("1~100 추측: "))
    if guess == answer:
        print("정답입니다!")
        break
    elif guess < answer:
        print("Up! ↑ 더 큰 수")
    else:
        print("Down! ↓ 더 작은 수")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

게임 업그레이드 (택1 이상)

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

따라 하기

- 시도 횟수 세기: 몇 번 만에 맞혔는지 출력 (count += 1)
- 기회 제한: 7번 안에 못 맞히면 실패 (if count > 7)

2. 값 바꾸기

3. 결과 설명

변형 미션

- 다시하기: 한 판 끝나면 "또 할래?(y/n)" 묻고 반복
- 숫자 합산기: 0 입력까지 숫자 받아 합계·평균 출력

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

휴식

휴식 10분

잠깐 멈추고, 다음 자료구조로 넘어갈 준비

10 MIN BREAK

다음 교시: 리스트 — 여러 값을 하나의 변수에 담는 마법 상자!

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장해 둡니다.

3교시

3교시 — 리스트

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: 리스트로 여러 데이터를 묶어 저장·수정·조회한다.

이론 20분

실습 30분

휴식 10분

이론

여러 값을 한 번에?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 학생 5명 점수를 저장하려면?
- 변수 100개는 비현실적
- 해결: 리스트 — 여러 값을 한 변수에!

Python example

```
score1 = 90  
score2 = 85  
score3 = 78  
score4 = 92  
score5 = 88 # 100명이면??
```

TAKEAWAY

학생 5명 점수를 저장하려면?

리스트란?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 리스트 = 여러 값을 순서대로 담는 상자
- 대괄호 [] 안에 콤마로 구분
- 하나의 이름으로 여러 값을 관리

Python example

```
scores = [90, 85, 78, 92, 88]
fruits = ["사과", "바나나", "포도"]
mixed = [1, "둘", 3.0, True] # 종류 섞여도 OK
```

TAKEAWAY

리스트 = 여러 값을 순서대로 담는 상자

리스트 생성

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- len() = 리스트 안 요소 개수
- 빈 리스트로 시작해 나중에 채우기도 가능 (반복문 + append)

Python example

```
empty = []          # 빈 리스트
numbers = [1, 2, 3, 4, 5]
names = ["김철수", "이영희"]
print(len(numbers))  # 5 (개수)
```

TAKEAWAY

len() = 리스트 안 요소 개수

이론

인덱싱 — 위치로 꺼내기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 인덱스는 0부터 시작! (1이 아님)
- [위치] 로 원하는 요소에 접근

Python example

```
fruits = ["사과", "바나나", "포도"]  
print(fruits[0]) # 사과 (첫 번째)  
print(fruits[1]) # 바나나  
print(fruits[2]) # 포도
```

TAKEAWAY

인덱스는 0부터 시작! (1이 아님)

이론

음수 인덱싱 — 뒤에서부터

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 음수는 뒤에서부터: -1=마지막, -2=뒤에서 둘째
- 마지막 요소를 쉽게 꺼낼 때 유용

Python example

```
fruits = ["사과", "바나나", "포도"]  
print(fruits[-1]) # 포도 (마지막)  
print(fruits[-2]) # 바나나
```

TAKEAWAY

음수는 뒤에서부터: -1=마지막, -2=뒤에서 둘째

이론

인덱스 범위 주의

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 요소가 3개면 인덱스는 0~2 (마지막은 len-1)
- 범위를 넘으면 IndexError
- 헛갈리면 len() 으로 개수 확인

Python example

```
fruits = ["사과", "바나나", "포도"] # 인덱스 0,1,2
print(fruits[3]) # IndexError!
```

TAKEAWAY

요소가 3개면 인덱스는 0~2 (마지막은 len-1)

이론

값 변경 (리스트는 가변)

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 리스트는 값을 바꿀 수 있음 (가변, mutable)
- 위치를 지정해 새 값 대입
- (어제 배운 문자열은 못 바꿨던 것과 대조)

Python example

```
fruits = ["사과", "바나나", "포도"]  
fruits[1] = "딸기"  
print(fruits) # ['사과', '딸기', '포도']
```

TAKEAWAY

리스트는 값을 바꿀 수 있음 (가변, mutable)

슬라이싱 — 잘라내기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- [시작:끝] → 시작 포함, 끝은 불포함 (range와 동일!)
- 여러 요소를 한 번에 잘라 가져오기

Python example

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4]) # [20, 30, 40]
```

TAKEAWAY

[시작:끝] → 시작 포함, 끝은 불포함 (range와 동일!)

이론

슬라이싱 — 생략 기법

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 시작 생략 = 처음부터, 끝 생략 = 끝까지
- [:] = 전체

Python example

```
nums = [10, 20, 30, 40, 50]
print(nums[:3]) # [10, 20, 30] (처음부터)
print(nums[2:]) # [30, 40, 50] (끝까지)
print(nums[:]) # 전체 복사
```

TAKEAWAY

시작 생략 = 처음부터, 끝 생략 = 끝까지

이론

슬라이싱 — 간격

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- `[::간격]` → 일정 간격으로
- `[::-1]` → 리스트 뒤집기 (유명한 기법!)

Python example

```
nums = [10, 20, 30, 40, 50]
print(nums[::2]) # [10, 30, 50] (2칸씩)
print(nums[::-1]) # [50, 40, 30, 20, 10] (거꾸로!)
```

TAKEAWAY

`[::간격]` → 일정 간격으로

이론

리스트에 추가 — append, insert

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- append = 맨 뒤, insert(위치, 값) = 원하는 위치

Python example

```
fruits = ["사과", "바나나"]  
fruits.append("포도")    # 맨 뒤 추가  
print(fruits) # ['사과', '바나나', '포도']
```

```
fruits.insert(0, "딸기")  # 0번 위치에 삽입  
print(fruits) # ['딸기', '사과', '바나나', '포도']
```

TAKEAWAY

append = 맨 뒤, insert(위치, 값) = 원하는 위치

리스트에서 삭제 — remove, pop

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- remove(값) = 그 값 삭제, pop() = 맨 뒤 꺼내기
- del fruits[0] = 위치로 삭제

Python example

```
fruits = ["사과", "바나나", "포도"]  
fruits.remove("바나나") # 값으로 삭제  
print(fruits) # ['사과', '포도']
```

```
last = fruits.pop() # 맨 뒤 꺼내며 삭제  
print(last) # 포도
```

TAKEAWAY

remove(값) = 그 값 삭제, pop() = 맨 뒤 꺼내기

이론

in — 포함 확인

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 값 in 리스트 → 들어있으면 True
- 검색·중복 체크에 유용 (조건문과 결합)

Python example

```
fruits = ["사과", "바나나", "포도"]  
print("사과" in fruits) # True  
print("수박" in fruits) # False
```

TAKEAWAY

값 in 리스트 → 들어있으면 True

이론

정렬과 뒤집기 — sort, reverse

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- sort() 정렬, reverse() 뒤집기

Python example

```
nums = [3, 1, 4, 1, 5, 9]
nums.sort()      # 오름차순 정렬
print(nums)      # [1, 1, 3, 4, 5, 9]
nums.sort(reverse=True) # 내림차순
print(nums)      # [9, 5, 4, 3, 1, 1]
nums.reverse()   # 순서 뒤집기
```

TAKEAWAY

sort() 정렬, reverse() 뒤집기

리스트 합치기 & 복사

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- + 두 리스트 연결, extend 뒤에 이어붙이기
- [:] 로 복사본 생성 (원본 보호)

Python example

```
a = [1, 2, 3]
b = [4, 5, 6]
print(a + b)    # [1, 2, 3, 4, 5, 6] (연결)

a.extend(b)     # a에 b를 이어붙임
print(a)        # [1, 2, 3, 4, 5, 6]

copy = a[:]     # 슬라이싱으로 복사본 만들기
```

TAKEAWAY

+ 두 리스트 연결, extend 뒤에 이어붙이기

이론

유용한 함수 — sum, max, min

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 합계·최고·최저·평균을 한 줄로!
- 직접 반복문 안 짜도 됨

Python example

```
scores = [90, 85, 78, 92, 88]
print(sum(scores))      # 433 (합계)
print(max(scores))      # 92 (최고)
print(min(scores))      # 78 (최저)
print(sum(scores) / len(scores)) # 86.6 (평균)
```

TAKEAWAY

합계·최고·최저·평균을 한 줄로!

실습

학생 점수 통계

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
scores = [85, 92, 78, 90, 88, 76, 95]

print(f"학생 수: {len(scores)}명")
print(f"총점: {sum(scores)}점")
print(f"평균: {sum(scores) / len(scores):.1f}점")
print(f"최고: {max(scores)}점")
print(f"최저: {min(scores)}점")

scores.sort(reverse=True)
print(f"순위: {scores}")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

To-do 리스트

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
todos = []
while True:
    print(f"\n현재 할 일: {todos}")
    cmd = input("추가(a)/삭제(d)/종료(q): ")
    if cmd == "a":
        todos.append(input("할 일: "))
    elif cmd == "d":
        todos.remove(input("지울 일: "))
    elif cmd == "q":
        break
    print("수고하셨습니다!")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

리스트 다루기 (택1 이상)

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
nums = [3, 7, 1, 9, 4, 6, 2]
```

실습 포인트

- 짝수만 골라 새 리스트 만들기 (반복문 + if + append)
- 평균보다 높은 점수만 출력
- 리스트를 거꾸로 출력 ([::-1])
- 가장 큰 값과 그 위치(인덱스) 찾기

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

휴식

휴식 10분

잠깐 멈추고, 다음 자료구조로 넘어갈 준비

10 MIN BREAK

다음 교시: 리스트 심화 & 튜플 — 한 줄로 리스트 만들기
, 못 바꾸는 리스트?

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장해 둡니다.

4교시

4교시 — 리스트 심화 & 튜플

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: 반복문과 리스트를 결합하고, 컴프리헨션·2차원 리스트·튜플을 익힌다.

이론 20분

실습 30분

휴식 10분

복습

리스트 + 반복문 (복습 강화)

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

어제의 한계

- 변수·입출력·연산자·조건문으로 한 번 판단하는 프로그램을 만들었습니다.
- 하지만 자판기처럼 한 번 실행하면 프로그램이 끝났습니다.
- 복사-붙여넣기는 느리고, 실수하고, 수정하기 어렵습니다.

오늘의 해결

- 반복문으로 여러 번 처리합니다.
- 리스트와 튜플로 여러 값을 묶습니다.
- 딕셔너리와 집합으로 이름표·중복·공통값을 다룹니다.

끝나면 만들 수 있는 것

- 구구단·별찍기 자동 출력, 숫자 맞추기 게임, 성적 통계, 단어장, 연락처 관리

TAKEAWAY

복붙 대신 반복문, 변수 여러 개 대신 자료구조로 갑니다.

이론

반복문으로 리스트 쌓기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 빈 리스트 → 반복하며 append 로 채우기
- "그릇 준비 → 쌓기" 패턴의 리스트 버전

Python example

```
squares = []  
for i in range(1, 6):  
    squares.append(i * i)  
print(squares) # [1, 4, 9, 16, 25]
```

TAKEAWAY

빈 리스트 → 반복하며 append 로 채우기

이론

enumerate — 번호와 값 동시에

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- enumerate = 인덱스 + 값을 함께 꺼냄
- 순위·번호 매길 때 편리
- 출력:

Python example

```
fruits = ["사과", "바나나", "포도"]  
for i, fruit in enumerate(fruits):  
    print(f"{i}번: {fruit}")
```

0번: 사과
1번: 바나나
2번: 포도

TAKEAWAY

enumerate = 인덱스 + 값을 함께 꺼냄

이론

리스트 컴프리헨션 ① 소개

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 리스트를 한 줄로 만드는 파이썬 특유의 문법
- [표현식 for 변수 in 범위]

Python example

```
# 일반 방식 (3줄)
squares = []
for i in range(1, 6):
    squares.append(i * i)

# 컴프리헨션 (1줄!)
squares = [i * i for i in range(1, 6)]
```

TAKEAWAY

리스트를 한 줄로 만드는 파이썬 특유의 문법

이론

리스트 컴프리헨션 ② 기본

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 각 요소를 변형해 새 리스트 생성

Python example

```
nums = [1, 2, 3, 4, 5]
doubled = [n * 2 for n in nums]
print(doubled) # [2, 4, 6, 8, 10]
```

```
words = ["hi", "bye"]
upper = [w.upper() for w in words]
print(upper) # ['HI', 'BYE']
```

TAKEAWAY

각 요소를 변형해 새 리스트 생성

이론

리스트 컴프리헨션 ③ 조건

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 뒤에 if 조건 → 조건 맞는 것만 골라 담기
- 필터링을 한 줄로!

Python example

```
nums = [1, 2, 3, 4, 5, 6]
evens = [n for n in nums if n % 2 == 0]
print(evens)  # [2, 4, 6]
```

TAKEAWAY

뒤에 if 조건 → 조건 맞는 것만 골라 담기

이론

리스트 컴프리헨션 ④ 예시 모음

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 핵심 개념을 예시로 먼저 이해합니다.
- 코드로 직접 확인하며 감을 잡습니다.

Python example

```
# 0~9 중 3의 배수
[n for n in range(10) if n % 3 == 0] # [0, 3, 6, 9]

# 문자열 길이 리스트
words = ["apple", "kiwi", "banana"]
[len(w) for w in words] # [5, 4, 6]

# 점수에 5점씩 가산
[s + 5 for s in [80, 90, 70]] # [85, 95, 75]
```

TAKEAWAY

핵심 개념을 예시로 먼저 이해합니다.

이론

일반 반복문 vs 컴프리헨션

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 둘 다 결과 같음: [2, 4]
- 짧은 변형/필터는 컴프리헨션, 복잡한 로직은 일반 반복문
- 억지로 한 줄에 욱여넣지 말 것! (읽기 어려우면 일반 반복문)

Python example

```
# 일반 방식 (4줄)
result = []
for n in range(1, 6):
    if n % 2 == 0:
        result.append(n)

# 컴프리헨션 (1줄)
result = [n for n in range(1, 6) if n % 2 == 0]
```

TAKEAWAY

둘 다 결과 같음: [2, 4]

이론

2차원 리스트 (리스트의 리스트)

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 리스트 안에 리스트 → 표·격자 표현
- [행][열] 로 접근

Python example

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(matrix[0])    # [1, 2, 3] (첫 줄)  
print(matrix[1][2]) # 6 (둘째 줄, 셋째 칸)
```

TAKEAWAY

리스트 안에 리스트 → 표·격자 표현

이론

2차원 리스트 + 중첩 for

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 바깥 for = 각 행, 안쪽 for = 행 안의 값
- 출력:

Python example

```
matrix = [[1, 2, 3], [4, 5, 6]]
for row in matrix:
    for value in row:
        print(value, end=" ")
    print() # 줄바꿈
```

```
1 2 3
4 5 6
```

TAKEAWAY

바깥 for = 각 행, 안쪽 for = 행 안의 값

이론

튜플(tuple)이란?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 튜플 = 리스트와 비슷하지만 소괄호 ()
- 여러 값을 묶는다는 점은 같음

Python example

```
point = (3, 5)
rgb = (255, 0, 128)
print(point[0]) # 3
```

TAKEAWAY

튜플 = 리스트와 비슷하지만 소괄호 ()

이론

튜플 vs 리스트 — 핵심 차이

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 리스트 [] = 바꿀 수 있음(가변)
- 튜플 () = 바꿀 수 없음(불변, immutable)

Python example

```
nums = [1, 2, 3] # 리스트: 변경 가능
nums[0] = 100   # OK

point = (1, 2, 3) # 튜플: 변경 불가!
point[0] = 100   # TypeError
```

TAKEAWAY

리스트 [] = 바꿀 수 있음(가변)

이론

튜플은 언제 쓰나?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 바뀌면 안 되는 값: 좌표 (x, y), RGB 색상, 요일 목록
- 실수로 변경 방지 → 더 안전
- 함수가 여러 값을 한 번에 돌려줄 때 (다음 주 함수에서!)

Python example

```
weekdays = ("월", "화", "수", "목", "금")
```

TAKEAWAY

바뀌면 안 되는 값: 좌표 (x, y), RGB 색상, 요일 목록

이론

튜플 언패킹

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 튜플을 여러 변수로 한 번에 풀기
- 변수 교환(swap)이 한 줄로!

Python example

```
point = (3, 5)
x, y = point    # 한 번에 나눠 담기
print(x, y)     # 3 5

# 변수 교환도 깔끔하게
a, b = 1, 2
a, b = b, a     # a=2, b=1
```

TAKEAWAY

튜플을 여러 변수로 한 번에 풀기

정리

리스트 ↔ 튜플 변환 & 정리

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

오늘의 결과물

- 필요에 따라 서로 변환 가능
- 정리: 리스트=가변[], 튜플=불변(), 컴프리헨션=한 줄 생성

다음 연결

- 함수(def)로 반복되는 코드를 이름 붙여 재사용
- 모듈로 남이 만든 기능 가져다 쓰기
- 문자열을 split, join으로 더 잘 다루기

TAKEAWAY

핵심은 반복해서 입력받고, 자료구조에 저장하고, 필요한 값을 분석하는 흐름입니다.

실습

로또 번호 생성기

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
import random

# 1~45 중 6개 중복 없이 뽑기
lotto = random.sample(range(1, 46), 6)
lotto.sort()
print(f"이번 주 행운의 번호: {lotto}")
```

실습 포인트

- random.sample(범위, 개수) = 중복 없이 뽑기
- sort() 로 오름차순 정렬

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

컴프리헨션 연습

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
# ① 1~10 제곱
print([n ** 2 for n in range(1, 11)])

# ② 1~20 중 3의 배수
print([n for n in range(1, 21) if n % 3 == 0])

# ③ 점수 합격(60+) 여부
scores = [55, 80, 45, 90, 70]
print([s >= 60 for s in scores])
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

리스트·튜플 종합 (택1 이상)

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

따라 하기

- 주사위 10번: 1~6 랜덤 10개 리스트로, 가장 많이 나온 눈은?
- 좌표 거리: 두 점 $(x1, y1)$, $(x2, y2)$ 튜플로 받아 거리 계산

2. 값 바꾸기

3. 결과 설명

변형 미션

- 이름 길이순 정렬: 이름 리스트를 길이 기준 정렬
- 컴프리헨션으로 구구단 2단 리스트 만들기

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

휴식

휴식 10분

잠깐 멈추고, 다음 자료구조로 넘어갈 준비

10 MIN BREAK

다음 교시: 딕셔너리 & 집합 — 이름표로 찾는 데이터, 중복 없는 모음!

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장해 둡니다.

5교시

5교시 — 딕셔너리 & 집합

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: 키-값 구조의 딕셔너리와 중복 없는 집합을 실생활 데이터에 적용한다.

이론 20분

실습 30분

휴식 10분

이론

리스트의 불편함

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 인덱스 [1] 이 무슨 값인지 기억해야 함
- 이름표가 있으면 좋겠다 → 딕셔너리!

Python example

```
# 사람 정보를 리스트로?  
person = ["홍길동", 24, "서울"]  
print(person[1]) # 24인데... 이게 뭐였지?
```

TAKEAWAY

인덱스 [1] 이 무슨 값인지 기억해야 함

이론

딕셔너리란?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 딕셔너리 = 이름표(키)로 값을 찾는 자료구조
- 중괄호 {} 안에 키: 값 쌍
- 사전(dictionary)처럼: 단어(키) → 뜻(값)

Python example

```
person = {"이름": "홍길동", "나이": 24, "도시": "서울"}  
print(person["이름"]) # 홍길동  
print(person["나이"]) # 24
```

TAKEAWAY

딕셔너리 = 이름표(키)로 값을 찾는 자료구조

이론

딕셔너리 생성

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 키: 보통 문자열(이름표 역할)
- 값: 무엇이든 (숫자, 문자열, 리스트...)
- 키는 중복 불가, 값은 중복 가능

Python example

```
empty = {}           # 빈 딕셔너리
scores = {"국어": 90, "영어": 85}
phone = {"엄마": "010-1111", "친구": "010-2222"}
```

TAKEAWAY

키: 보통 문자열(이름표 역할)

값 접근하기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 딕셔너리[키] 로 값을 꺼냄
- 리스트는 위치([0]), 딕셔너리는 이름표(["수학"])
- 없는 키를 부르면 KeyError

Python example

```
scores = {"국어": 90, "영어": 85, "수학": 95}  
print(scores["수학"]) # 95
```

TAKEAWAY

딕셔너리[키] 로 값을 꺼냄

이론

값 추가 & 수정

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 없는 키에 대입 → 추가
- 있는 키에 대입 → 수정(덮어쓰기)

Python example

```
scores = {"국어": 90}
scores["영어"] = 85    # 새 키 추가
scores["국어"] = 100   # 기존 키 수정
print(scores)  # {'국어': 100, '영어': 85}
```

TAKEAWAY

없는 키에 대입 → 추가

이론

값 삭제

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- del 또는 pop() 으로 삭제

Python example

```
scores = {"국어": 90, "영어": 85, "수학": 95}
del scores["영어"]    # 키로 삭제
print(scores)         # {'국어': 90, '수학': 95}

removed = scores.pop("수학") # 꺼내며 삭제
print(removed)         # 95
```

TAKEAWAY

del 또는 pop() 으로 삭제

이론

get() — 안전하게 접근

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 없는 키여도 에러 없이 None(또는 기본값)
- ["키"] 는 에러, .get() 은 안전
- 빈도수 세기 등에서 필수!

Python example

```
scores = {"국어": 90, "영어": 85}
print(scores.get("수학"))      # None (에러 X)
print(scores.get("수학", 0))   # 0 (기본값 지정)
```

TAKEAWAY

없는 키여도 에러 없이 None(또는 기본값)

이론

in — 키 존재 확인

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 키 in 딕셔너리 → 키가 있으면 True
- 접근 전에 미리 확인하면 안전

Python example

```
scores = {"국어": 90, "영어": 85}
print("국어" in scores) # True
print("수학" in scores) # False
```

TAKEAWAY

키 in 딕셔너리 → 키가 있으면 True

이론

keys / values / items

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- keys() 키 모음, values() 값 모음, items() 쌍 모음
- 반복문과 결합해 전체 순회에 사용

Python example

```
scores = {"국어": 90, "영어": 85}
print(scores.keys()) # dict_keys(['국어', '영어'])
print(scores.values()) # dict_values([90, 85])
print(scores.items()) # [('국어', 90), ('영어', 85)]
```

TAKEAWAY

keys() 키 모음, values() 값 모음, items() 쌍 모음

이론

딕셔너리 순회 (for)

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- items() + 튜플 언패킹으로 키·값 동시에
- 출력:

Python example

```
scores = {"국어": 90, "영어": 85, "수학": 95}
for subject, score in scores.items():
    print(f"{subject}: {score}점")
```

```
국어: 90점
영어: 85점
수학: 95점
```

TAKEAWAY

items() + 튜플 언패킹으로 키·값 동시에

이론

딕셔너리 활용 — 빈도수 세기

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- `get(ch, 0)` 으로 처음엔 0, 나올 때마다 +1
- 글자·단어 빈도, 투표 집계 등에 단골

Python example

```
text = "banana"
count = {}
for ch in text:
    count[ch] = count.get(ch, 0) + 1
print(count) # {'b': 1, 'a': 3, 'n': 2}
```

TAKEAWAY

`get(ch, 0)` 으로 처음엔 0, 나올 때마다 +1

이론

집합(set)이란?

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- 집합 = 중복 없는 값들의 모음, 중괄호 { }
- 딕셔너리도 { } → 집합은 키:값 없이 값만
- 순서가 없음 (인덱싱 불가)

Python example

```
numbers = {1, 2, 3, 2, 1}
print(numbers)  # {1, 2, 3} (중복 사라짐!)
```

TAKEAWAY

집합 = 중복 없는 값들의 모음, 중괄호 { }

이론

집합의 특징 — 중복 제거

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- set() 으로 리스트의 중복을 한 방에 제거
- 다시 list() 로 변환 가능
- 가장 실용적인 활용!

Python example

```
nums = [1, 2, 2, 3, 3, 3, 4]
unique = set(nums)
print(unique)          # {1, 2, 3, 4}
print(list(unique))    # [1, 2, 3, 4]
```

TAKEAWAY

set() 으로 리스트의 중복을 한 방에 제거

이론

집합 연산 — 합집합·교집합

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

개념 포인트

- | 합집합(전부), & 교집합(공통)
- 수학의 집합 개념 그대로!

Python example

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)   # {1,2,3,4,5,6} 합집합
print(a & b)   # {3, 4} 교집합 (공통)
```

TAKEAWAY

| 합집합(전부), & 교집합(공통)

정리

집합 연산 — 차집합 & 정리

개념을 짧게 이해하고, 바로 코드로 반복·자료구조를 체험

오늘의 결과물

- - 차집합 (한쪽에만 있는 것)
- 정리: 딕셔너리=키:값 {}, 집합=중복 없는 값 {}
- 집합 활용: 중복 제거, 공통/차이 찾기

다음 연결

- 함수(def)로 반복되는 코드를 이름 붙여 재사용
- 모듈로 남이 만든 기능 가져다 쓰기
- 문자열을 split, join으로 더 잘 다루기

TAKEAWAY

핵심은 반복해서 입력받고, 자료구조에 저장하고, 필요한 값을 분석하는 흐름입니다.

실습

영어 단어장

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
words = {"apple": "사과", "book": "책", "cat": "고양이"}

while True:
    word = input("\n검색할 단어(끝: q): ")
    if word == "q":
        break
    meaning = words.get(word, "단어장에 없습니다")
    print(f"뜻: {meaning}")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

글자 빈도수 세기

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
text = input("문장 입력: ")
count = {}
for ch in text:
    if ch == " ":
        continue      # 공백 제외
    count[ch] = count.get(ch, 0) + 1

for ch, n in count.items():
    print(f"{ch}: {n}번")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

실습

집합으로 분석 (택1 이상)

반복 → 저장 → 분석 흐름을 직접 실행하며 확인

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
class_a = {"김철수", "이영희", "박민수", "최지우"}  
class_b = {"이영희", "박민수", "정해인", "한소희"}
```

실습 포인트

- 공통 수강생 찾기 (&)
- A반에만 있는 학생 (-)
- 전체 학생 명단 (|)
- 중복 포함 리스트에서 중복 제거(set)

TAKEAWAY

반복과 자료구조는 직접 값을 바꿔 실행할 때 가장 빨리 익숙해집니다.

휴식

오프라인 정리 & 휴식

잠깐 쉬고, 온라인에서는 "현실 데이터"를 다뤄봅시다!

10 MIN BREAK

반복문(for/while) · break/continue

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장해 둡니다.